



COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the *Copyright Act 1968* (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.



FIT3080 – Artificial Intelligence

Propositional and
First Order Logic
Sections 7.1, 7.3-7.5,
8.1-8.2, 9.1-9.2, 9.5

Knowledge Representation

- **How do we represent facts about the world?**
- **How do we reason about these facts?**
- **Some widely accepted formal calculi**
 - Probability calculus
 - **Propositional logic**
 - **First-order logic**



Assumptions about the Environment

- **Observable**
- **Known**
- **Single/multi agent**
- **Deterministic**
- **Sequential/episodic**
- **Static**
- **Discrete**



What is Logic?

- **A method for storing information in a knowledge base and reasoning with it**
 - Syntax – how to express this knowledge
 - Semantics – what these expressions mean
- **A method for deriving conclusions from known facts**
 - Proof system – a way of manipulating expressions to get other expressions and/or draw conclusions



Knowledge Representation: Learning Objectives

- **Knowledge-based agents**
 - **Logic in general – models and entailment**
 - **Propositional logic**
 - **Equivalence, validity, satisfiability**
 - **First-order logic**
 - **Inference rules and theorem proving**
 - forward chaining
 - backward chaining
 - resolution refutation systems
 - > substitution
 - > unification
 - > resolution
- Propositional logic
- First order logic

A Simple Knowledge-based Agent

```
function KB-AGENT(percept) returns an action
  static: KB, a knowledge base
         t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

- **The agent must be able to:**
 - Represent states, actions, etc
 - Incorporate new percepts
 - Update internal representations of the world
 - Deduce hidden properties of the world
 - Deduce appropriate actions





FIT3080 – Artificial Intelligence

Propositional Logic

Propositional Logic – Syntax (I)

- **Atomic sentence:** a symbol, e.g., P
- **Literal:** an atomic sentence or its negation
 - E.g., P , $\neg P$
- **Connectives**
 - conjunction, e.g., $\neg A \wedge B$
 - disjunction, e.g., $A \vee B$
 - implication, e.g., $A \Rightarrow B$
 - biconditional, e.g., $A \Leftrightarrow B$
- **Clause:** a **disjunction** of literals
 - E.g., $\neg P \vee Q \vee A$
- **Conjunctive Normal Form (CNF):** a conjunction of a finite set of clauses
 - E.g., $(\neg P \vee Q) \wedge (Q \vee A)$

Propositional Logic – Syntax (II)

Sentence $\rightarrow$ *AtomicSentence* | *ComplexSentence*

AtomicSentence $\rightarrow$ *True* | *False* | *P* | *Q* | *R* | ...

ComplexSentence $\rightarrow$ (*Sentence*)

| $\neg$ *Sentence*

| *Sentence* $\wedge$ *Sentence*

| *Sentence* $\vee$ *Sentence*

| *Sentence* $\Rightarrow$ *Sentence*

| *Sentence* $\Leftrightarrow$ *Sentence*

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$



Models

- **Models** are a formal representation of worlds with respect to which truth can be evaluated
- A model of a sentence is an assignment of a truth value to every symbol
 - E.g., $m(x_1) = T$, $m(x_2) = F$, $m(x_3) = F$
- If a sentence α is true in a model m , we say that
 - m is a model of α , or m satisfies α
 - E.g., m satisfies the sentence $(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3)$
- If no model satisfies a sentence, then the sentence is unsatisfiable

Semantics

- **Each model specifies true/false for each proposition**
 - E.g.,

P	Q	R
false	true	false
- **Rules for evaluating truth with respect to a model m :**
 - $\neg P$ is true iff P is false in m
 - $P \wedge Q$ is true iff P is true and Q is true in m
 - $P \vee Q$ is true iff P is true or Q is true in m
 - $P \Rightarrow Q \equiv \neg P \vee Q$ is true iff P is false or Q is true in m
 - $P \Leftrightarrow Q$ is true iff $P \Rightarrow Q$ is true and $Q \Rightarrow P$ is true in m
- **A simple recursive process evaluates an arbitrary sentence**
 - E.g., $\neg P \wedge (Q \vee R) = \text{true} \wedge (\text{true} \vee \text{false}) = \text{true} \wedge \text{true} = \text{true}$



Truth Tables for Connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true



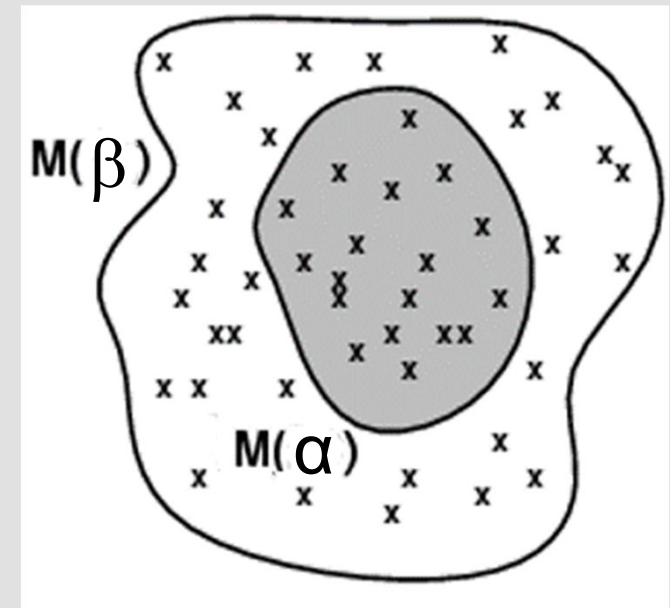


FIT3080 – Artificial Intelligence

Theorem Proving

Logical Entailment

- Entailment means that one sentence **follows logically** from another: $\alpha \models \beta$
sentence α entails sentence β iff β is true in all models where α is true
 - $M(\alpha)$ = the set of all models of α
 - $\alpha \models \beta$ iff $M(\alpha) \subseteq M(\beta)$
 - > E.g., α = Giants won & Reds won
 β = Giants won
- α is the stronger assertion
- Entailment is a relationship between sentences (i.e., **syntax**) that is based on **semantics**



Proving that a Sentence α is True

- **Model checking**

Enumerate all possible models to check that a sentence α is true in all models where KB is true

- **Truth table enumeration** (2^n , where n is the number of symbols)
- **Improved backtracking** (recursive enumeration of all models) with logic-related heuristics

- **Application of inference procedures**

Sound generation of new sentences from old

- **Proof** = a sequence of inference rule applications
 - > use inference rules as operators in a standard search algorithm
 - > typically require transformation of sentences into a ***normal form***

Inference Procedures

- $KB \vdash_i \alpha$ means that sentence α can be derived from KB by procedure i
- **Soundness:** a sound inference procedure i derives only entailed sentences –
if $KB \vdash_i \alpha$, then $KB \models \alpha$
- **Completeness:** a complete inference procedure i can derive any sentence that is entailed –
if $KB \models \alpha$, then that $KB \vdash_i \alpha$
- First-order logic (FOL) is expressive enough to say almost anything of interest, and there exists a sound and complete inference procedure for it

Grounding

The connection between logical reasoning processes and real environments

- **How do we know that KB is true in the real world?**
 - The agent's sensors create a connection, i.e., the meaning and truth of percept sentences are defined by the processes of sensing and sentence construction
- **Where do we get the rest of an agent's knowledge?**
 - By learning (generalizing) from experience

Concepts in Theorem Proving

- **Logical equivalences**
- **Validity and satisfiability**
- **Monotonicity**



Logical Equivalences

- Two sentences are **logically equivalent** iff they are both true in the same models: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$



Validity and Satisfiability (I)

- **A sentence is valid if it is true in all models**
 - E.g., *True*, $A \vee \neg A$, $A \Rightarrow A$
- **Validity is connected to inference via the Deduction Theorem**
 - $\alpha \models \beta$ iff $\alpha \Rightarrow \beta$ is valid
- **A sentence is satisfiable if it is true in some model**
 - E.g., $A \vee B$, C
- **A sentence is unsatisfiable if it is true in no model**
 - E.g., $A \wedge \neg A$

Validity and Satisfiability (II)

- **Validity and satisfiability are connected**
 - α is valid iff $\neg\alpha$ is unsatisfiable
 - α is satisfiable iff $\neg\alpha$ is not valid
 - $\alpha \models \beta$ iff $\alpha \wedge \neg\beta$ is unsatisfiable

Proving β from α by checking the unsatisfiability of $\alpha \wedge \neg\beta$ corresponds to the mathematical proof technique of *reductio ad absurdum* aka proof by refutation or proof by contradiction

Monotonicity

- The set of entailed sentences can only increase as information is added to the knowledge base
- For any sentences α and β ,
if $KB \models \alpha$ then $KB \wedge \beta \models \alpha$



PL – Common Rules of Inference

- **Modus Ponens**

$$\frac{\alpha \Rightarrow \beta , \alpha}{\beta}$$

- **Modus Tollens**

$$\frac{\alpha \Rightarrow \beta , \neg \beta}{\neg \alpha}$$

- **And Elimination**

$$\frac{\alpha \wedge \beta}{\alpha}$$

- **Resolution**

$$\frac{x \vee \alpha , \neg x \vee \beta}{\alpha \vee \beta}$$

Parent clauses

Resolvent



Resolution – Some Intuition

- $x \vee \alpha$ is equivalent to $\neg x \Rightarrow \alpha$ and $\neg \alpha \Rightarrow x$
- $\neg x \vee \beta$ is equivalent to $x \Rightarrow \beta$ and $\neg \beta \Rightarrow \neg x$
- Chaining these implications:
 $\neg \alpha \Rightarrow x$ and $x \Rightarrow \beta$ gives $\neg \alpha \Rightarrow \beta$,
which is equivalent to $\alpha \vee \beta$



Common Rules of Inference – Example

Deductive inference is the process of deriving new facts from existing facts

- **Facts in the KB:**

- $p_1: \neg \text{Rained} \Rightarrow \text{Harry visited Hagrid}$
- $p_2: \text{Harry visited Dumbledore}$
- $p_3: \text{Harry visited Dumbledore} \Rightarrow \neg \text{Harry visited Hagrid}$
- $Q: \text{Did it rain?}$

- **Proof:**

- $p_2 \ \& \ p_3 \text{ (Modus Ponens)} \Rightarrow p_4: \neg \text{Harry visited Hagrid}$
- $p_4 \ \& \ p_1 \text{ (Modus Tollens)} \Rightarrow p_5: \neg \neg \text{Rained} \Rightarrow \text{Rained}$



Proof as Search

- **Initial state:** initial KB
- **Action:** apply an inference rule to the sentences that match the parent clauses of the rule
- **Result:** add the resolvent of the rule to the KB
- **Goal:** a state where the KB contains the sentence we are trying to prove

Monotonicity: the set of entailed sentences can only increase as information is added to the KB

Horn Clauses and Definite Clauses

- **Definite clause** – a disjunction of literals of which exactly one is positive
 - E.g., $\neg A \vee \neg B \vee C$
 - Definite clauses can be written as implications $(A \wedge B) \Rightarrow C$
- **Horn clause** – a disjunction of literals of which at most one is positive
 - All definite clauses are Horn clauses
 - Inference with Horn clauses can be done with **forward** or **backward** chaining
 - Deciding entailment with Horn clauses can be done in time that is linear on the size of the KB



Backward Chaining

- **IDEA: Work backwards from the query Q :**
 - to prove Q by Backward Chaining, check if Q is known already, or prove by Backward Chaining all premises of some rule concluding Q
- **Avoid loops: check if new subgoal is already on the goal stack**
- **Avoid repeated work: check if a new subgoal**
 1. has already been proved true, or
 2. has already failed

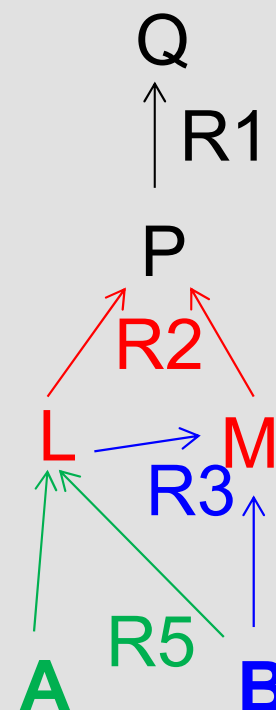
Backward chaining is sound and complete for Horn KBs

Backward Chaining – Example

- **KB:**

- R1: $P \Rightarrow Q$
- R2: $L \wedge M \Rightarrow P$
- R3: $B \wedge L \Rightarrow M$
- R4: $A \wedge P \Rightarrow L$
- R5: $A \wedge B \Rightarrow L$
- A
- B

- **Prove Q**



Forward Chaining Algorithm

- **IDEA: Work forwards from the facts in KB**

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
           q, the query, a proposition symbol
  count  $\leftarrow$  a table, where count[c] is initially the number of symbols in clause c's premise
  inferred  $\leftarrow$  a table, where inferred[s] is initially false for all symbols
  queue  $\leftarrow$  a queue of symbols, initially symbols known to be true in KB

  while queue is not empty do
    p  $\leftarrow$  POP(queue)
    if p = q then return true
    if inferred[p] = false then
      inferred[p]  $\leftarrow$  true
      for each clause c in KB where p is in c.PREMISE do
        decrement count[c]
        if count[c] = 0 then add c.CONCLUSION to queue
  return false
```

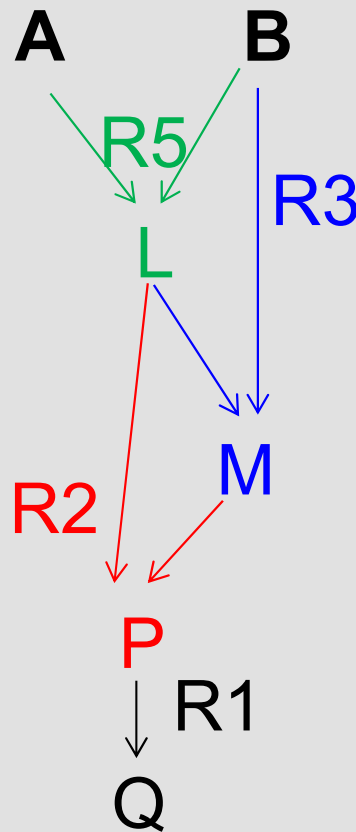
Forward chaining is sound and complete for Horn KBs

Forward Chaining – Example

- **KB:**

- R1: $P \Rightarrow Q$
- R2: $L \wedge M \Rightarrow P$
- R3: $B \wedge L \Rightarrow M$
- R4: $A \wedge P \Rightarrow L$
- R5: $A \wedge B \Rightarrow L$
- A
- B

- **Prove Q**



Queue	Count					Inferred
	R1	R2	R3	R4	R5	
A B	1	2	2	2	2	



Forward versus Backward Chaining

- **Forward Chaining**
 - **data-driven**, automatic, unconscious processing
 - > e.g., object recognition, routine decisions
 - may do lots of work that is irrelevant to the goal
- **Backward Chaining**
 - **goal-driven**, appropriate for problem-solving
 - > e.g., How do I get into a PhD program?
 - complexity can be **much less** than linear in size of KB
- **Both are sound and complete for Horn KBs**

Resolution

- **Resolution** – an inference rule applied to clauses that yields a complete inference algorithm when coupled with any complete search algorithm

$$\begin{array}{c} \text{parent clauses} \\ l_1 \vee \dots \vee \textcolor{red}{l_i} \vee \dots \vee l_k \qquad m_1 \vee \dots \vee \textcolor{red}{m_j} \vee \dots \vee m_n \\ \hline l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n \\ \text{resolvent} \end{array}$$

- where l_i and m_j are **complementary literals** ($l_i = \neg m_j$)
- the resolvent should contain only one copy of each literal
- **Resolution**
 - is sound for propositional logic
 - applies only to clauses

Conversion to Conjunctive Normal Form

- Every sentence in propositional logic is logically equivalent to a conjunction of clauses
- Converting a sentence to Conjunctive Normal Form (CNF)
 1. Eliminate $\Leftrightarrow$: replace $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
 2. Eliminate $\Rightarrow$: replace $\alpha \Rightarrow \beta$ with $\neg \alpha \vee \beta$
 3. Move $\neg$ inwards by repeated application of the following equivalences:
 - > double-negation: $\neg (\neg \alpha) \equiv \alpha$
 - > de Morgan $\neg(\alpha \wedge \beta) \equiv \neg \alpha \vee \neg \beta$
 - > de Morgan $\neg(\alpha \vee \beta) \equiv \neg \alpha \wedge \neg \beta$
 4. Apply distributive law ($\wedge$ over $\vee$) and flatten



Conversion to CNF – Example

- **Convert to CNF:** $(B \vee C) \Rightarrow A$
 1. Eliminate $\Rightarrow$: $\neg (B \vee C) \vee A$
 2. Move $\neg$ inwards: $(\neg B \wedge \neg C) \vee A$
 3. Apply distributive law: $(\neg B \vee A) \wedge (\neg C \vee A)$



Resolution-refutation Systems

- $\alpha \models \beta$ iff $\alpha \wedge \neg\beta$ is unsatisfiable
- **Proof by refutation**
 1. Negate the goal and add the negation to the set of clauses
 2. Apply resolution to the clauses in the set of clauses until a contradiction is reached
 - $\alpha \wedge \neg\beta$ are unsatisfiable



Resolution-refutation Algorithm

- **Proof by contradiction, i.e., given a goal α , show that $KB \wedge \neg\alpha$ is unsatisfiable**

function PL-RESOLUTION(KB, α) **returns** *true* or *false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$

$new \leftarrow \{ \}$

while *true* **do**

for each pair of clauses C_i, C_j **in** $clauses$ **do**

$resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)

if $resolvents$ contains the empty clause **then return** *true*

$new \leftarrow new \cup resolvents$

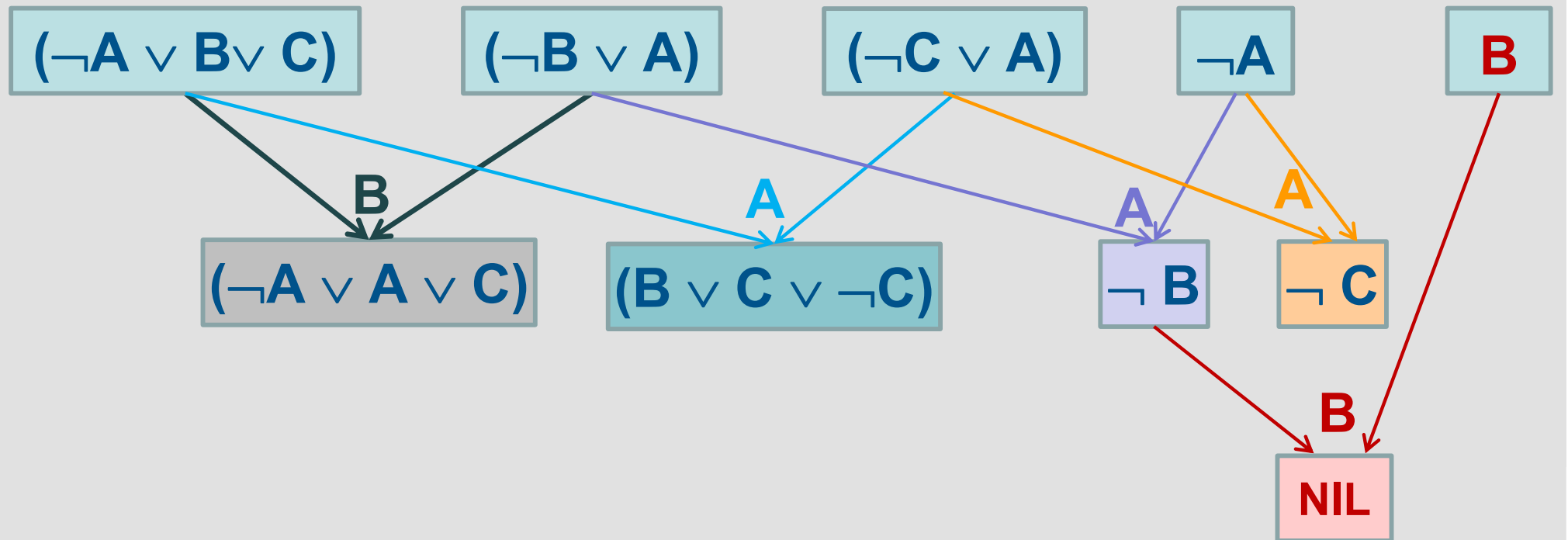
if $new \subseteq clauses$ **then return** *false*

$clauses \leftarrow clauses \cup new$



Resolution-refutation – Example

- $KB = (A \Leftrightarrow (B \vee C)) \wedge \neg A$
 $(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A) \wedge \neg A$
- Prove: $\alpha = \neg B$



Soundness and Completeness

Resolution refutation is sound and complete for propositional logic

- **Resolution Closure** $RC(S)$ of a set of clauses S is the set of all clauses derivable by repeated application of the resolution rule to the clauses in S or their derivatives
 - $RC(S)$ is finite $\rightarrow$ Resolution always terminates
- **Ground resolution theorem:** if a set of clauses is unsatisfiable, then the resolution closure of these clauses contains the empty clause

Pros and Cons of Propositional Logic

😊 **declarative**

😊 **allows partial/disjunctive/negated information**

😊 **compositional:**

– meaning of $A \wedge B$ is derived from meaning of A and of B

😊 **Meaning is context-independent**

😞 **Propositional logic has very limited expressive power**

– E.g., cannot talk about groups of things



FIT3080 – Artificial Intelligence

First Order Logic

First-order Logic (FOL)

- **First-order logic assumes the world contains**
 - **Objects**: people, houses, numbers, colors, ...
 - **Relations**: red, round, prime, bigger than, ...
 - **Functions**: father of, best friend, plus, ...
- **FOL has increased expressive power**



First Order Logic – Syntax (I)

- **Term:** constant, variable or function
- **Atomic formula or atom:** predicate symbol and terms
 predicate constant variable
– e.g., MARRIED(*John*,*Mother(x)*)
 function
- **Literal:** atomic formula or negation of an atomic formula
- **Ground literal:** literal without variables,
– e.g., AUNT(*Mary,Sue*)
- **Connectives**
 - disjunction, e.g., $P(x) \vee Q(y) \vee W(x,y)$
 - conjunction, e.g., $P(x) \wedge Q(y)$
 - implication, e.g., $P(x) \Rightarrow W(x,y) \equiv \neg P(x) \vee W(x,y)$
 - bi-conditional, e.g., $P(x) \Leftrightarrow W(x,y)$

First Order Logic – Syntax (II)

- **Clause:** disjunction of literals
 - e.g., $P(x) \vee Q(y, f(x))$
- **Conjunctive Normal Form (CNF):** a conjunction of a finite set of clauses
 - e.g., $(\neg P1(x) \vee P2(y)) \wedge P3(x)$

First Order Logic – Quantification

- **Quantification**

- Universal ($\forall$) –

All elephants are gray predicate constant

$\forall x [\text{ELEPHANT}(x) \Rightarrow \text{COLOUR}(x, \text{Gray})]$
variable

- Existential ($\exists$) –

Someone wrote “Computer Chess”

$\exists x \text{ WRITE}(x, \text{Computer-Chess})$

- **Nested quantification**

- Existential inside the scope of universal

E.g., $\forall s \exists c \text{ Eats}(s, c)$

- Universal inside the scope of existential

E.g., $\exists c \forall s \text{ Eats}(s, c)$



First Order Logic – Syntax (III)

$Sentence \rightarrow AtomicSentence \mid ComplexSentence$

$AtomicSentence \rightarrow Predicate \mid Predicate(Term, \dots) \mid Term = Term$

$ComplexSentence \rightarrow (Sentence)$

$\mid \neg Sentence$

$\mid Sentence \wedge Sentence$

$\mid Sentence \vee Sentence$

$\mid Sentence \Rightarrow Sentence$

$\mid Sentence \Leftrightarrow Sentence$

$\mid Quantifier Variable, \dots Sentence$



FOL – Some Equivalences

- $\neg \exists x P(x) \equiv \forall x \neg P(x)$

There does not exist an x such that $P(x)$ is true $\equiv$
For all x , $P(x)$ is false

- $\neg \forall x P(x) \equiv \exists x \neg P(x)$

It is not true that for all x $P(x)$ is true $\equiv$
There exists an x , such that $\neg P(x)$

- $\forall x [P(x) \wedge Q(x)] \equiv \forall x P(x) \wedge \forall y Q(y)$

For all x , $P(x)$ and $Q(x)$ are true $\equiv$
For all x , $P(x)$ is true, and for all y $Q(y)$ is true

- $\exists x [P(x) \vee Q(x)] \equiv \exists x P(x) \vee \exists y Q(y)$

There is an x , such that $P(x)$ is true or $Q(x)$ is true $\equiv$
There is an x , such that $P(x)$ is true, or
there is a y , such that $Q(y)$ is true

FOL Rules of Inference – Example

- **Modus Ponens**

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

- **Universal Specialization (where A is a constant)**

$$\frac{\forall x W(x)}{W(A)}$$

- **Universal Specialization + Modus Ponens**

1. All dogs bark: $\forall x [\text{DOG}(x) \Rightarrow \text{BARKS}(x)]$

2. Fido is a dog: $\text{DOG}(\text{FIDO})$

Prove that Fido barks: $\text{BARKS}(\text{FIDO})$

Universal specialization: $\text{DOG}(\text{FIDO}) \Rightarrow \text{BARKS}(\text{FIDO})$

Modus ponens: $\text{BARKS}(\text{FIDO})$

General Inference – Resolution Refutation

- **Resolution**
 - Unification
 - > Substitution
 - Converting sentences to clauses



Substitution

- **Substitution is a set of ordered pairs**
 $s = \{v_1|t_1, v_2|t_2, \dots, v_n|t_n\}$
where $v_i|t_i$ means that term t_i substitutes variable v_i throughout
 - Example:
 $P(x,y) \{x|A, y|B\} \rightarrow P(A,B)$
- **Semantics: all elements are applied *simultaneously***
 - Example:
 $P(w, y, f(x)) \{y|h(x), x|g(y)\} \rightarrow P(w, h(x), f(g(y)))$



Composition of Substitutions

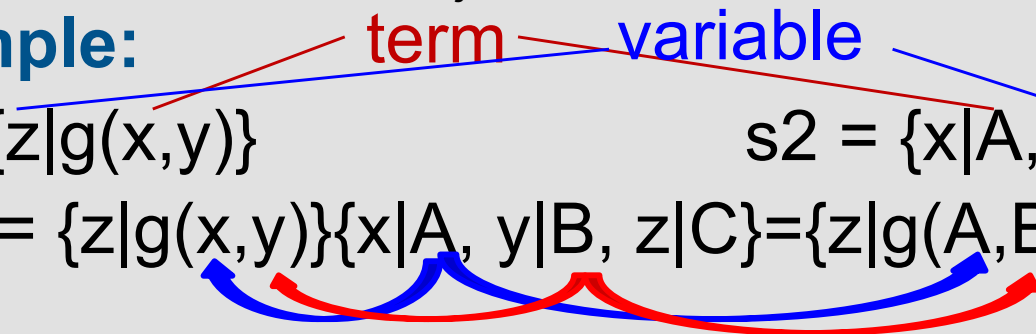
- **$s_i s_j$ – composition of two substitutions**

- Apply s_j to the terms of s_i
- Add any pairs of s_j having variables **not** in s_i

Example:

$s_1 = \{z|g(x,y)\}$ $s_2 = \{x|A, y|B, z|C\}$

$s_1 s_2 = \{z|g(x,y)\}\{x|A, y|B, z|C\} = \{z|g(A,B), x|A, y|B\}$



$s_2 s_1 = \{x|A, y|B, z|C\} \{z|g(x,y)\} = \{x|A, y|B, z|C\}$

- **Properties of compositions of substitutions**

- $L(s_1 s_2) = (L s_1) s_2$
- Associative: $(s_1 s_2) s_3 = s_1 (s_2 s_3)$
- NOT commutative: $s_1 s_2 \neq s_2 s_1$

Unification

- A process that finds substitutions of terms for variables, such that two expressions are identical
- A set $\{E_i\}$ of expressions is unifiable, if there exists a substitution s such that $E_1s = E_2s = \dots$
In this case, s is a unifier of $\{E_i\}$
- **mgu (most general unifier)** – the mgu g of $\{E_i\}$ has the property that if s is any unifier of $\{E_i\}$, yielding $\{E_i\}s$, then there exists a substitution s' such that $\{E_i\}s = \{E_i\}gs'$
- Example:
 $s = \{x|A, y|B\}$ unifies $P(x, f(y))$ with $P(x, f(B))$
but the mgu is $\{y|B\}$

Algorithm Unify (list-structured expressions) Nilsson

Algorithm Unify(E1,E2)

1. If either E1 or E2 is a variable or symbol, then interchange E1 and E2 if necessary, so that E1 is a variable or symbol
 - a. If E1 and E2 are identical then return { } // no substitution
 - b. If E1 is a variable do
 - i. If E1 occurs in E2 then return FAIL
 - ii. return {E1|E2} // E2 may be a compound expression
 - c. If E2 is a variable then return {E2|E1}
 - d. return FAIL
2. F1 ← the first element of E1, T1 ← rest of E1
3. F2 ← the first element of E2, T2 ← rest of E2
4. Z1 ← Unify(F1,F2)
5. If Z1 = FAIL, then return FAIL
6. G1 ← result of applying Z1 to T1; G2 ← result of applying Z1 to T2
7. Z2 ← Unify(G1,G2)
8. If Z2 = FAIL, then return FAIL
9. return the composition of Z1 and Z2

predicate,
function,
negation,
constant

Unification Example: $\text{Unify}(P(y, g(y)), P(z, g(x)))$

Unify((P y (g y)), (P z (g x)))

- 1a Unify (P, P): P and P are identical $\rightarrow \emptyset$
- 4 **Unify ((y (g y)), (z (g x)))**
- 1b Unify(y, z) $\rightarrow \{y|z\}$
- 6 Apply $\{y|z\}$ to the rest of the lists $\rightarrow (g z), (g x)$
- 1a Unify (g, g): g and g are identical $\rightarrow \emptyset$
- 4 **Unify((z), (x))**
- 1b Unify(z, x) $\rightarrow \{z|x\}$
- 9 return composition of $\{y|z\}\{z|x\} = \{y|x, z|x\}$



Converting sentences to Clauses

1. Eliminate implication symbols
2. Reduce scopes of negation symbols
3. **Standardize variables (for each quantifier)**
4. **Eliminate existential quantifiers (*skolemize*)**
5. **Move all universal quantifiers to the front**
6. Put result in conjunctive normal form (CNF)
7. **Eliminate universal quantifiers**
8. **Eliminate $\wedge$ symbols**
9. **Rename variables (standardize variables apart for each clause)**

FOL – Nesting of Quantifiers & Skolemization

- **Existential inside the scope of universal**

E.g., $\forall s \exists c \text{ Eats}(s, c)$

Skolem function: $\forall s \text{ Eats}(s, g(s))$

- **Universal inside the scope of existential**

E.g., $\exists c \forall s \text{ Eats}(s, c)$

Skolem constant: $\forall s \text{ Eats}(s, A)$

Skolemization replaces existentially quantified variables with functions or constants, depending on the nesting

Converting Sentences to Clauses – Example 1

Whoever can read is intelligent

$\forall x [\text{CanRead}(x) \Rightarrow \text{Intelligent}(x)]$

1. Eliminate $\Rightarrow$: $\forall x [\neg \text{CanRead}(x) \vee \text{Intelligent}(x)]$

7. Eliminate $\forall$: $\neg \text{CanRead}(x) \vee \text{Intelligent}(x)$



Converting Sentences to Clauses – Example 2

$$\forall x [\neg \forall y [P(x,y) \Rightarrow Q(x,y)]]$$

1. Eliminate $\Rightarrow$: $\forall x [\neg \forall y [\neg P(x,y) \vee Q(x,y)]]$

2. Reduce scope of $\neg$: $\forall x [\exists y \neg [\neg P(x,y) \vee Q(x,y)]]$
 $\forall x \exists y [P(x,y) \wedge \neg Q(x,y)]$

4. Eliminate $\exists$: $\forall x [P(x,g(x)) \wedge \neg Q(x,g(x))]$

7. Eliminate $\forall$: $P(x,g(x)) \wedge \neg Q(x,g(x))$

8. Eliminate $\wedge$ symbols: $\{ P(x,g(x)), \neg Q(x,g(x)) \}$

9. Standardize variables apart: $\{ P(x_1,g(x_1)), \neg Q(x_2,g(x_2)) \}$

General Resolution

- Let the prospective parent clauses be $\{L_j\}$ and $\{M_i\}$ (with variables standardized apart)

parent clauses

$$l_1 \vee \dots \vee l_i \vee \dots \vee l_k$$

$$m_1 \vee \dots \vee m_j \vee \dots \vee m_n$$

$$l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n$$

resolvent

- where $\{l_i\}$ is a subset of $\{L_j\}$ and $\{m_j\}$ is a subset of $\{M_i\}$ such that an mgu s exists for the sets $\{l_i\}$ and $\{\neg m_j\}$

- The clauses $\{L_j\}$ and $\{M_i\}$ **resolve** and the new clause $\{\{L_j\} - \{l_i\}\}s \cup \{\{M_i\} - \{m_j\}\}s$ is their **resolvent**

General Resolution – Example

1. Everyone who can read is literate

$$\forall x [\text{CANREAD}(x) \Rightarrow \text{LITERATE}(x)]$$

2. Whoever goes to school can read

$$\forall x [\text{GOSCHOOL}(x) \Rightarrow \text{CANREAD}(x)]$$

$$1. \neg \text{GOSCHOOL}(x_1) \vee \underline{\text{CANREAD}(x_1)}$$

$$2. \neg \underline{\text{CANREAD}(x_2)} \vee \text{LITERATE}(x_2)$$

$$\text{mgu } s = \{x_2/x_1\}$$

$$3. \neg \text{GOSCHOOL}(x_1) \vee \text{LITERATE}(x_1)$$

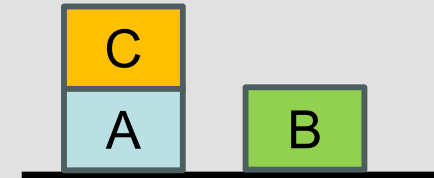


Resolution-refutation Systems

- $\alpha \models \beta$ iff $\alpha \wedge \neg\beta$ is unsatisfiable
- **Proof by refutation**
 1. Negate the goal and add the negation to the set of clauses
 2. Apply resolution to the clauses in the set of clauses until a contradiction is reached
 - $\alpha \wedge \neg\beta$ are unsatisfiable

Theorem Proving – Example (I)

1. $ON(C,A)$
2. $ONTABLE(A)$
3. $ONTABLE(B)$
4. $CLEAR(C)$
5. $CLEAR(B)$
6. If a block is clear, there is nothing on it
 $\forall x [CLEAR(x) \Rightarrow \neg \exists y ON(y,x)]$
7. **Goal:** Prove that there is nothing on C
 $\neg \exists y ON(y,C)$



Theorem Proving – Example (II)

1. ON(C,A)
2. ONTABLE(A)
3. ONTABLE(B)
4. CLEAR(C)
5. CLEAR(B)



6. $\forall x [\text{CLEAR}(x) \Rightarrow \neg \exists y \text{ ON}(y,x)]$

Eliminate implication: $\forall x [\neg \text{CLEAR}(x) \vee \neg \exists y \text{ ON}(y,x)]$

Reduce scope of negation: $\forall x [\neg \text{CLEAR}(x) \vee \forall y \neg \text{ON}(y,x)]$

Move $\forall$ to the front and remove: **6'. $\neg \text{CLEAR}(x) \vee \neg \text{ON}(y,x)$**

7. **Goal:** Prove $\neg \exists y \text{ ON}(y,C)$

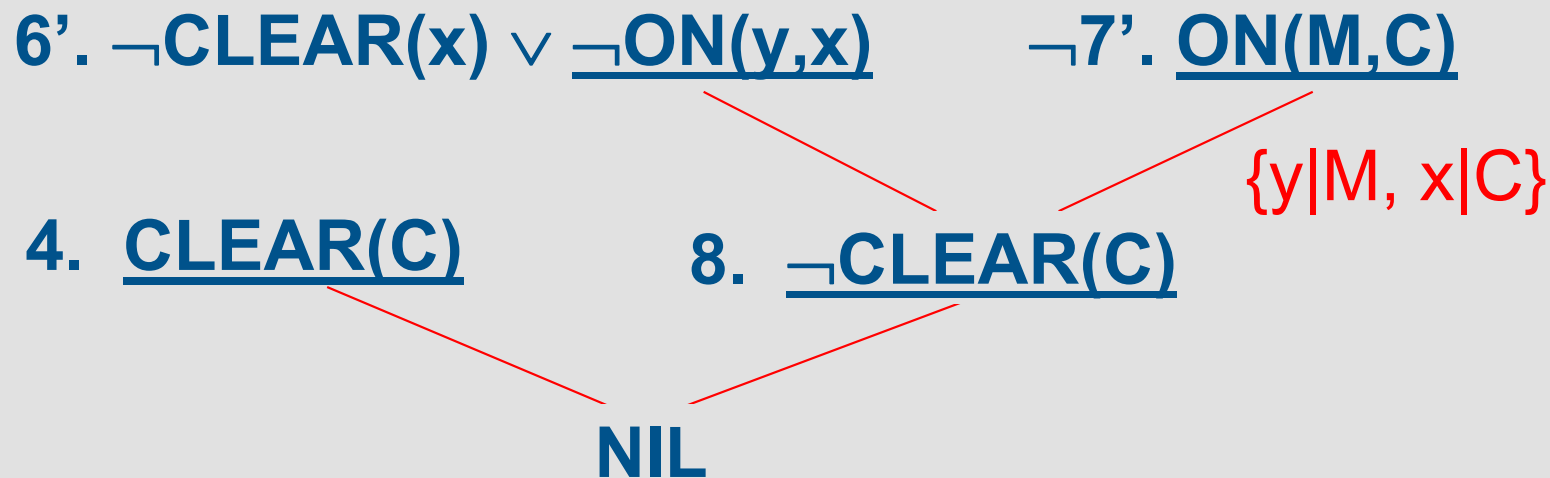
Negate the goal: $\neg 7: \neg \neg \exists y \text{ ON}(y,C) \equiv \exists y \text{ ON}(y,C)$

Skolemize: **$\neg 7'. \text{ON}(M,C)$**

$\text{KB} = \{ 1, 2, 3, 4, 5, 6', \neg 7' \}$

Theorem Proving – Example (III)

1. $\text{ON}(\text{C}, \text{A})$
2. $\text{ONTABLE}(\text{A})$
3. $\text{ONTABLE}(\text{B})$
4. $\text{CLEAR}(\text{C})$
5. $\text{CLEAR}(\text{B})$
- 6'. $\neg \text{CLEAR}(\text{x}) \vee \neg \text{ON}(\text{y}, \text{x})$
- $\neg$ 7'. $\text{ON}(\text{M}, \text{C})$



Resolution-refutation Strategies

- **Unit preference:** prefer resolutions where one of the sentences is a **unit clause** (single literal)
 - **Unit resolution:** every resolution *must* involve a unit clause
- **Set of support:** every resolution step should involve at least one element of a special set of clauses, the **set of support**. The resolvent is added to the set of support
 - E.g., set of support = $\{\neg Q\}$
- **Input resolution:** every resolution combines one of the original input sentences (from the KB or the query) with another sentence
- **Subsumption:** eliminates all sentences that are subsumed by (more specific than) an existing sentence in the KB

Applications of First-order Logic

- **Automated theorem proving**
- **Software verification**
 - How do we verify that a program does what it was designed to do?
- **Question answering**
- **Planning**
- **Natural language understanding**

Reading

- **Russell, S. and Norvig, P., *Artificial Intelligence – A Modern Approach* (4th ed), Prentice Hall**
 - Chapter 7, Sections 7.1-7.5
 - Chapter 8, Sections 8.1-8.3
 - Chapter 9, Sections 9.1, 9.2 and 9.5



Next Lecture Topic

- **Lecture Topic 5**
 - Probability and Bayesian Networks

